# GFAz: State-of-the-Art Graphical Fragment Assembly Compression

Taolue Yang
taolue.yang@temple.edu
Temple University
Philadelphia, PA, USA

Youyuan Liu
youyuan.liu@temple.edu
Temple University
Philadelphia, PA, USA

Bo Jiang
bo.jiang@temple.edu
Temple University
Philadelphia, PA, USA

Xinghua Shi
mindyshi@temple.edu
Temple University
Philadelphia, PA, USA

Sian Jin
sian.jin@temple.edu
Temple University
Philadelphia, PA, USA

## Abstract

Pangenome graphs stored in the Graphical Fragment Assembly (GFA) format can reach hundreds of gigabytes, with traversal records accounting for over 90% of the file size. General-purpose compressors fail to exploit cross-haplotype redundancy in these traversals, and existing specialized tools either sacrifice full semantics or offer limited throughput, while still trailing our method in compression ratio. We present GFAz, a lossless GFA compressor that compresses and decompresses in linear time. GFAz uses iterative 2-mer grammar compression to collapse traversal redundancy, combined with columnar, type-specific encoding of all GFA record types including optional fields. Both compression and decompression are fully parallel, with a CPU backend (OpenMP) and a GPU backend (CUDA/nvComp). Across six datasets ranging from 113 MB to 369 GB, GFAz achieves 18–84× compression, outperforming all evaluated baselines in compression ratio, compression throughput, and decompression throughput. The CPU backend sustains up to 385 MB/s compression and 1.8 GB/s decompression; the GPU backend reaches up to 4.8 GB/s and 9.4 GB/s, respectively.

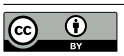
## CCS Concepts

• **Theory of computation** → **Data compression**; • **Applied computing** → *Bioinformatics*; • **Computing methodologies** → *Parallel algorithms.*

## Keywords

Biological Data Compression, Graphical Fragment Assembly

**ACM Reference Format:**
Taolue Yang, Youyuan Liu, Bo Jiang, Xinghua Shi, and Sian Jin. 2026. GFAz: State-of-the-Art Graphical Fragment Assembly Compression. In *2026 International Conference on Supercomputing (ICS '26), July 06–09, 2026, Belfast, United Kingdom.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3797905.3807870



This work is licensed under a Creative Commons Attribution 4.0 International License.
*ICS '26, Belfast, United Kingdom*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2522-7/2026/07
https://doi.org/10.1145/3797905.3807870

## 1 Introduction

Human genomes vary across individuals at many scales, from small variants to large structural changes, yet most analysis still relies predominantly on mapping sequencing reads to a single linear reference genome. While effective in conserved regions, this paradigm breaks down in highly polymorphic loci and in sequences absent from the reference, leading to systematic reference bias [14]. Pangenome graphs [3, 17] address this limitation by representing population variation explicitly within a unified structure: nodes encode DNA segments, edges capture admissible adjacencies, and paths represent individual haplotypes. By eliminating reliance on a single reference, graph-based representations substantially improve downstream analyses, including up to 34% fewer small-variant errors and more than doubling structural-variant discovery compared to linear-reference approaches [13].

Recently, the Graphical Fragment Assembly (GFA) format [6] has emerged as the de facto interchange format for pangenome graphs. GFA is simple, human-readable, and widely adopted by graph construction and analysis pipelines such as Minigraph-Cactus [9] and PGGB [4]. However, this generality of GFA comes at a cost of scale. While graph topology grows sublinearly as haplotypes are added, traversal records grow nearly linearly because each traversal is materialized explicitly. This structural redundancy rapidly dominates file size, making large GFA files expensive to store, transfer, and process, and thus motivating the need for effective file compression.

Real pangenome datasets make this bottleneck explicit. Sirén and Paten [20] report that 90-haplotype human pangenome graphs constructed from year-1 HPRC data require 44.9 GiB and 88.6 GiB as uncompressed GFA text for Cactus and PGGB, respectively; even after gzip compression, they remain 11.1 GiB and 14.6 GiB. At larger cohort scale, their 5008-haplotype 1000GP graph reaches 9534.9 GiB uncompressed and 2231.3 GiB after gzip compression. This growth is largely driven by traversal data: in a human chromosome 19 graph with 1,000 haplotypes, traversals alone account for 98.7% of a 14 GB GFA file [8].

In practice, gzip-compatible compressors such as gzip and bgzip remain widely used in bioinformatics pipelines because they are ubiquitous, stable, and interoperable with existing Unix tooling. However, pangenome graphs expose a mismatch between general-purpose compression and graph structure. Gzip-style compressors operate over local byte windows and are not designed to exploit

repeated traversal structure across chromosome-scale haplotypes, leaving substantial cross-haplotype redundancy unexploited.

Two specialized approaches illustrate the current design trade-offs. GBZ [20] achieves weaker compression on highly collapsed regions and small graphs, and is not designed for incremental updates. In contrast, sqz [8] preserves a text workflow but is relatively slow in practice due to its heap-based Byte Pair Encoding (BPE) design. Both approaches do not preserve all original GFA record information, such as optional fields, which limits their use as fully lossless GFA text compressors.

We thus present GFAz, a GFA compressor that achieves state-of-the-art compression ratio and high throughput on both CPU and GPU. In summary, GFAz makes four main contributions:

- **Orientation-aware grammar compression**: multi-round 2-mer grammar that exploits the canonical symmetry of forward/reverse node orientations with linear-time compression and decompression.
- **Columnar GFA representation**: column-oriented layout for all GFA record types with type-specific encodings, enabling individual field access without full decompression.
- **Incremental haplotype extension and random access**: support for adding extra haplotypes without recompressing previously compressed data, and for retrieving individual haplotypes without decompressing the full dataset.
- **Fully parallel CPU and GPU backends**: end-to-end parallel compression and decompression using OpenMP and CUDA, with a shared format so that data compressed on CPU can be decompressed on GPU for faster decoding.

Across large human pangenome datasets, GFAz achieves 18–84× compression, typically over 10× better than gzip/zstd baselines. CPU throughput reaches 200–400 MB/s for compression and 2 GB/s for decompression; the GPU backend reaches up to 5 GB/s and 10 GB/s, respectively. These results make GFAz practical for large-scale pangenome storage and transfer. We release our implementation as open source at: https://github.com/babyplutokurt/GFAz.

## 2 Background

### 2.1 The GFA Format

GFA is a line-oriented text format for sequence graphs [6]. Biologically, a graph node represents a DNA sequence segment (often shared by many individuals), edges represent allowed adjacencies between segments, and traversals represent assembled sequences or haplotypes as ordered walks through those segments.

The node record is **S** (segment), which stores a segment identifier and its DNA sequence. Edge records include **L** (link), **J** (jump), and **C** (containment): **L** connects oriented segments (with overlap information), **J** represents long-range connections or gaps, and **C** represents containment/nesting relationships. Traversal records are **P** (path) and **W** (walk). Both describe an ordered, orientation-aware route through nodes, which reconstructs a longer biological sequence from graph segments. **P** is a general named traversal (often a contig or reference path), while **W** (GFA 1.1) represents haplotype walks with explicit sample, haplotype, and coordinate metadata. **H** (header) stores file-level metadata, and optional `TAG:TYPE:VALUE` fields extend records with additional annotations. Figure 1 shows a visualization of a GFA file using Bandage [22].

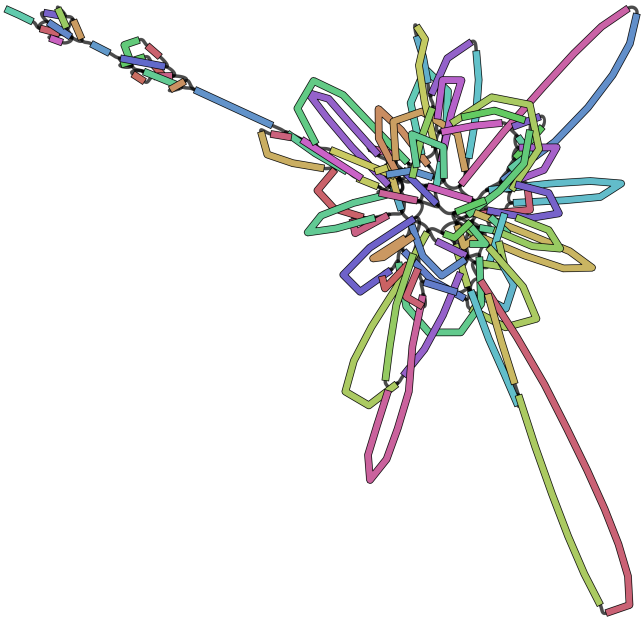


**Figure 1: Assembly graph in GFA format, visualized with Bandage [22]. Colored elements denote sequence segments (nodes; S records). Black connections indicate edges between oriented segments, represented by L (link), J (jump), or C (containment) records. Any valid traversal through connected segments corresponds to a P (path) or W (walk) record.**

### 2.2 The Scalability Challenge

GFA is widely used in graph-based genomics, including assembly graphs, pangenome construction, and downstream analysis. In pangenome graphs, shared sequence is merged into common nodes and variants form alternate branches. HPRC releases have grown from tens to hundreds of haplotypes and continue to expand [11, 21]. Many downstream tools operate directly on GFA graphs, so these large files create both memory pressure during analysis and substantial storage overhead. At this scale, GFA becomes a systems bottleneck rather than only a storage nuisance.

As highlighted in Section 1, real pangenome GFA files already reach very large sizes. The core scaling mechanism is structural: topology (S/L/J/C records) grows sublinearly as haplotypes are added, but traversal records (P/W) grow near-linearly because each haplotype walk is materialized explicitly. Since human haplotypes are highly similar, these long traversal streams contain substantial repeated node-ID substrings across samples. At even larger population sequencing scales, this growth will continue, pushing GFA datasets beyond hundreds gigabytes into multi-terabyte regimes. In practice, pipelines also produce many intermediate GFAs (per chromosome, per version, and per region), further amplifying storage and transfer costs.

Generic compressors cannot exploit most of this redundancy. Their small sliding windows miss long-range similarity across chromosome-length paths, so compression ratio remains limited for GFA files.

**Table 1: Mapping from GFA record types to GFAz internal integer-based representation.**

| GFA Record | Graph Role | Internal Representation |
|---|---|---|
| S-line | Node Definition | `uint32_t` (1-based ID) |
| L, J, C-lines | Graph Topology | `vector<int32_t>` |
| P, W-lines | Graph Traversal | `vector<vector<int32_t>>` |

## 2.3 Existing Compression Approaches

Because generic compressors miss long-range path redundancy, specialized GFA compressors are needed. Other genomic compression families are also not a direct fit: read/alignment formats such as BAM/CRAM [1, 12] target mapped reads rather than graph interchange files, and FASTQ compressors such as GPUFastQLZ [23] target linear sequence collections rather than graph topology and traversal semantics. For GFA graphs, compression must remain fully lossless to preserve graph structure and metadata while still exploiting cross-haplotype traversal redundancy. Two graph-native approaches are GBZ and sqz.

GBZ uses Graph-Based Burrows–Wheeler Transform (GBWT) [18, 19] to store large collections of similar paths efficiently. This design is effective for large pangenomes with many similar haplotypes. Its weaknesses are practical. Performance can degrade in highly collapsed repetitive regions, and compression ratio is less favorable on small graphs. Because GBWT construction is global, GBZ also does not naturally support incremental updates (e.g., appending new haplotypes) without rebuilding the index. In addition, GBZ does not preserve full GFA record semantics, like optional fields.

sqz keeps a text workflow and applies a heap-based BPE grammar design [24] to path records. It repeatedly selects frequent adjacent 2-mers and replaces them with new rules. The main limitations are speed and memory cost. After each new rule, sqz updates frequencies of affected neighboring pairs through adjacency-list bookkeeping, which makes each rule-creation step expensive in practice. This sequential dependency also limits parallelization, because each next substitution depends on the current global pair statistics. Similar to GBZ, sqz does not preserve full optional-field information, so it is not truly lossless for complete GFA records.

# 3 Methodology

This section presents GFAz as a modularly designed compression pipeline with two interchangeable backends: CPU (OpenMP) and GPU (CUDA/Thrust/nvComp) [16]. The final entropy-encoding layer is also modular, so it can be changed or tuned easily to trade compression ratio against throughput.

## 3.1 Graph View of GFA

A GFA file is a tab-delimited graph format with a small set of record types. GFAz converts these heterogeneous text records into a compact Structure-of-Arrays layout of typed columns. Segments (S-lines) receive compact 1-based integer IDs, with sequences stored separately. Links, jumps, and containments (L/J/C-lines) become integer endpoint columns plus bit-packed orientation columns, with auxiliary strings stored as concatenated byte streams and offsets. Most importantly, both paths and walks (P/W-lines) are normalized to the same signed `int32_t` traversal representation, where the sign

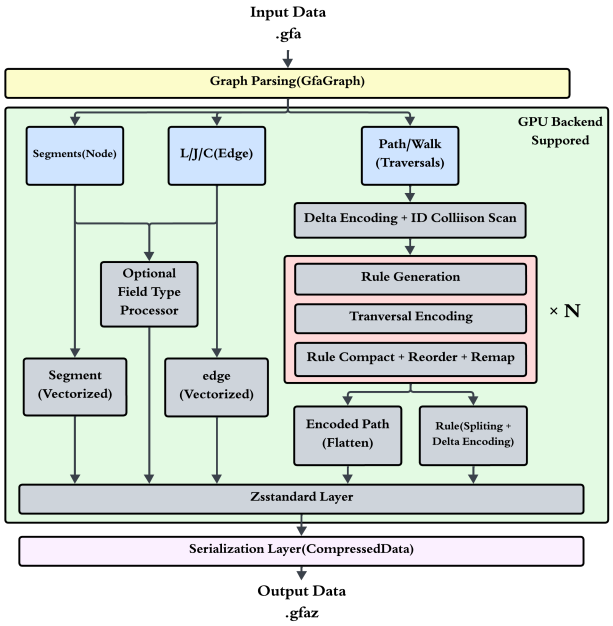



**Figure 2: GFAz compression workflow. The grammar compression loop repeats for $N$ rounds (default 8); each round replaces repeated 2-mers, reducing the traversal stream by up to half.**

encodes orientation; for example, `>s1<s2>s3` becomes `[1, -2, 3]`. This conversion turns verbose GFA text into compressible integer and byte streams, with traversals forming the dominant input to the grammar-compression pipeline.

Table 1 summarizes how the text-to-integer mapping prepares each component for the compression pipeline.

## 3.2 Architecture Overview

Figure 2 and 3 show the end-to-end workflow. A GFA file is first parsed into an in-memory graph representation: on the CPU this is a `GfaGraph` with nested vectors; for the GPU backend the traversals are flattened into a single contiguous array paired with a lengths vector, with offsets computed on-demand via parallel prefix sum. The parsed graph is then compressed into a `CompressedData` object containing byte blocks for every field, and can be serialized to a binary `.gfaz` file for persistent storage. Decompression is intentionally asymmetric: fixed-width graph fields are restored from their encoded columns, but traversal records are expanded and emitted as GFA text in a streaming fashion rather than rebuilding a full in-memory `GfaGraph`.

Both compression and decompression are fully parallel at every step. As introduced above, the pipeline supports CPU and GPU execution; the GPU backend keeps traversal data device-resident throughout the compression loop, transferring only the final compressed blocks back to the host.

## 3.3 Compression Pipeline

The central challenge in GFA compression is traversal data (P and W records), which often exceeds 90% of file size. Traversals are

**Algorithm 1** GFAz Compression Pipeline

**Require:** GFA file $F$, rounds $R$, delta_rounds $D$
**Ensure:** Compressed data $C$

```
G ← Parse(F)
for i = 1 to D do
    DeltaTransform(G.paths)
    DeltaTransform(G.walks)
end for
next_id ← max(MaxAbs(G.paths), |G.segments|) + 1
for r = 1 to R do
    rules ← GenerateRules(G.paths, G.walks)
    EncodePaths(G.paths, G.walks, rules)
    CompactSortRemap(rules, G.paths, G.walks)
end for
C ← CompressAllFields(G, rules)
return C
```

highly redundant: many haplotypes share long node-ID substrings across P-lines and W-lines.

GFAz exploits this structure with iterative grammar compression. In each round, we find frequent adjacent 2-mers (initially node IDs), assign each 2-mer a new integer ID, and replace all occurrences. Later rounds can combine original nodes with rule IDs, or combine rule IDs with each other. This hierarchy collapses long repeated patterns into single symbols and sharply reduces traversal length.

**Rule Definition.** We formally define a *rule* as a new unique integer identifier assigned to a repeated 2-mer. Like segment IDs, rule IDs are signed 32-bit integers where the sign encodes orientation. If a rule $r$ ($r > 0$) represents the 2-mer $(u, v)$, then its negation $-r$ represents the reverse complement 2-mer $(-v, -u)$. This symmetry stems from the fact that traversing a subgraph in reverse flips the order and orientation of all constituents.

$$r \Rightarrow (u, v) \quad \Longleftrightarrow \quad -r \Rightarrow (-v, -u) \tag{1}$$

By treating rules as oriented symbols identical in format to node IDs, the compression engine can recursively combine nodes and rules without special cases.

*3.3.1 Phase 1: Delta Encoding and Rule Namespace Allocation.* We begin traversal compression with a single delta-encoding pass. For a sequence $[n_1, n_2, \ldots, n_k]$, we produce $[n_1, n_2 - n_1, n_3 - n_2, \ldots, n_k - n_{k-1}]$ and then run grammar compression on this transformed stream. This step is effective because pangenome graph builders often assign segment IDs in approximate reference order. Many haplotypes therefore contain long runs of locally consecutive IDs, and one delta pass collapses these runs into short repeated differences. For GFAz, the main benefit is not merely smaller values, but a much smaller adjacent-pair space for rule mining: many distinct 2-mers in the original traversal stream become the same few repeated 2-mers after delta encoding. Table 2 shows this effect on a chromosome-scale graph: one delta pass reduces the first-round rule search space by over 25× and gives the best final size.

We therefore use exactly one delta pass throughout the system. Additional delta rounds do not improve compression ratio, because later rule IDs no longer preserve the sequential locality of original segment IDs.

**Table 2: Effect of delta encoding rounds on rule search space and final traversal size for `chr6.pan.gfa` (human chromosome 6 pangenome GFA). All results use 8 grammar-compression rounds.**

| Delta rounds | Rules (round 1) | Total rules | Final size |
|---|---|---|---|
| 0 | 5,455,923 | 29,391,077 | 3.60% |
| 1 | 201,503 | 6,124,767 | 2.88% |
| 2 | 264,485 | 6,281,692 | 2.99% |
| 3 | 372,451 | 7,053,633 | 3.11% |

To separate literals from grammar rules, we reserve the rule namespace above all literal values. We set the starting ID to:

$$I_{start} = \max\left(\max_{p \in \mathcal{P}} \max_{v \in p} |v|,\ |S|\right) + 1 \tag{2}$$

where the first term covers all values in the delta-encoded streams and $|S|$ covers valid segment IDs. This guarantees that a single check ($|x| \geq I_{start}$) uniquely identifies a rule.

*3.3.2 Phase 2: Hierarchical 2-Mer Grammar Compression.* The engine of our pipeline is a multi-round algorithm that recursively replaces frequent 2-mers with the rule IDs defined in the previous section. A *2-mer* in this context is simply an ordered pair of signed integers $(u, v)$, representing adjacent elements in a traversal.

Since GFA paths represent double-stranded DNA, a traversal $(u, v)$ and its reverse complement $(-v, -u)$ should map to the same rule. We therefore normalize every observed 2-mer to a *canonical form* by selecting the lexicographically smaller of its forward and reverse representations.

$$\text{pack}(u, v) = ((int64)u \ll 32) \mid (v \ \&\ \texttt{0xFFFFFFFF}) \tag{3}$$

$$\text{canonical}(u, v) = \min\left(\text{pack}(u, v), \text{pack}(-v, -u)\right) \tag{4}$$

By hashing and counting only these canonical values, occurrences on either strand contribute to the same rule. Each rule therefore corresponds to exactly one canonical 2-mer, while the sign of the emitted rule ID records whether the observed occurrence matches the canonical orientation or its reverse.

**Rule Generation.** To identify 2-mers that appear at least twice, we use a parallel "two-set" strategy that avoids building a full frequency table. Each thread scans adjacent pairs, inserts first occurrences into a local *seen* set, and promotes second occurrences to a local *repeated* set. The thread-local sets are then reduced into global *seen* and *repeated* sets, so a 2-mer observed once in multiple threads is still promoted to *repeated*. Paths and walks are processed together in this pass, ensuring that a 2-mer appearing once in a path and once in a walk is still detected as repeated. Finally, we assign sequential rule IDs to the unique canonical 2-mers in the global *repeated* set.

Suppose $n$ unique repeated 2-mers are found. Each is assigned a rule ID sequentially: the first receives start_id, the second start_id + 1, and so on up to start_id+$n$−1. Two structures are built: a hash map from canonical 2-mer to rule ID for $O(1)$ lookup during encoding, and a vector of length $n$ indexed by (rule_id − start_id) for $O(1)$ reverse lookup during later expansion.

**Traversal Encoding and Rule Tracking.** We encode each traversal by scanning left-to-right, checking every consecutive 2-mer against the rule hash map. For each 2-mer $(u, v)$, we first check if

**Table 3: Internal columnar data structures for each GFA record type before final ZSTD compression.**

| GFA Record | Component Fields | Internal Data Structure |
|---|---|---|
| **Rules** | Dictionary (First/Second) | Two parallel `vector<int32_t>` |
| | Layer Ranges | `vector<LayerRuleRange>` |
| **Segments (S)** | Sequences | Concatenated `string` + Lengths (`vector<uint32_t>`) |
| | Optional Fields | Type-specific columns (varint, float, string) |
| **Links (L)** | From/To Node IDs | Parallel `vector<uint32_t>` |
| | Orientations | Bit-packed `vector<uint8_t>` (1 bit/edge) |
| | Overlaps | Parallel `vector<uint32_t>` / `vector<char>` |
| | Optional Fields | Type-specific columns |
| **Jumps (J)** | From/To Node IDs | Parallel `vector<uint32_t>` (delta+varint) |
| | Orientations | Bit-packed `vector<uint8_t>` |
| | Distance | Concatenated `string` + Lengths |
| | Rest Fields | Concatenated `string` + Lengths |
| **Containments (C)** | Container/Contained IDs | Parallel `vector<uint32_t>` (delta+varint) |
| | Orientations | Bit-packed `vector<uint8_t>` |
| | Position | `vector<uint32_t>` (ZSTD) |
| | Overlap (CIGAR) | Concatenated `string` + Lengths |
| | Rest Fields | Concatenated `string` + Lengths |
| **Paths (P)** | Encoded Sequence | Flattened `vector<int32_t>` (Rules/IDs) |
| | Path Names, Overlaps | Concatenated `string` + Lengths |
| | Path Lengths | `vector<uint32_t>` (Original & Compressed) |
| **Walks (W)** | Encoded Sequence | Flattened `vector<int32_t>` (Rules/IDs) |
| | Sample/Seq IDs | Concatenated `string` + Lengths |
| | Haplotype/Start/End | `vector<uint32_t>` + `vector<int64_t>` (varint) |
| | Walk Lengths | `vector<uint32_t>` |

its canonical form corresponds to a known rule ID $r$. If so, we check orientation: if $(u, v)$ matches the canonical order, we replace the 2-mer with $+r$; if it matches the reverse complement order $(-v, -u)$, we replace it with $-r$.

Because rules are generated from the pre-encoding stream, newly emitted rule IDs cannot participate in additional same-round matches, so encoding remains a simple left-to-right scan.

During this process, we maintain a usage bitmap, `rules_used`, to track which generated rules are actually applied. This is necessary because greedy substitution may leave some generated rules unused; whenever a rule $r$ is emitted, we set `rules_used`$[|r| -$ start_id$] = 1$.

**Compact, Sort, and Remap.** After encoding, we optimize the rule set for storage and the next grammar round via three fused operations:

(1) **Compact (Prune Unused):** As noted, greedy encoding leaves gaps in the rule index space. To remove them, we perform an *exclusive prefix sum* on the `rules_used` bitmap. The result at index $i$ gives the exact destination index for the rule at (start_id + $i$) in the new particular array. For example, if `rules_used` is $[1, 0, 1, 1]$, the prefix sum is $[0, 1, 1, 2]$. This tells us: rule 0 moves to pos 0, rule 1 is skipped, rule 2 moves to pos 1, and rule 3 moves to pos 2. We then gather only the active rules into a value-contiguous array using these computed offsets, completely eliminating gaps in parallel.

(2) **Sort (Optimize Layout):** We sort all the active rules by their packed 2-mer content. This step is crucial for the compression of the rulebook itself. By ordering rules lexicographically (primarily by their first element $u$, secondarily by $v$), we cluster rules that share the same starting node. For storage, each packed 64-bit rule key is split into its upper 32 bits and lower 32 bits. After sorting, the upper-32-bit stream becomes highly repetitive (e.g., $[\ldots, u, u, u, w, w, \ldots]$), so delta encoding produces long zero runs and sharply reduces rulebook size.

(3) **Remap (Update Streams):** Finally, we assign permanent, sequential IDs to the rules in their new sorted order, starting from the current global `next_id`. We verify the permutation by building a lookup table, $T[\text{old\_id} - \text{range\_start}] = \text{new\_id}$. A parallel function then sweeps over all paths: any element $e$ falling in the current round's range is updated to $T[|e| - \text{range\_start}]$ (preserving the original sign). Upon completion, we update `start_id` to be the last assigned rule ID +1, ensuring that rules in the next grammar round begin exactly where the current set ends, which results in sequential rule IDs across all rounds, ranging from the first round's start ID to the last end ID.

The Generation-Encoding-Compact, Sort and Remap phases constitute a single round. We repeat this process iteratively (typically 8 times). As described earlier, subsequent rounds can pair the rules created in previous steps, allowing the algorithm to capture exponentially longer patterns (length 4, 8, 16, and beyond) and recursively collapse long repetitive haplotype blocks into single rule IDs. After these rounds, the traversal data consists of encoded traversal streams, a global rulebook stored as parallel `rules_first` and `rules_second` vectors, and the boundary value `start_id` that separates literals from rule IDs during decoding. This representation exposes both traversals and rules as integer streams for the final field-specific encoding phase.

*3.3.3 Phase 3: Field-Specific Encoding.* After Phases 1–2, GFAz stores the graph as typed streams. Phase 3 converts all records to a columnar Structure-of-Arrays (SoA) layout: one contiguous column per field. Each column then uses a type-specific encoder:

- **Integers (IDs, lengths, positions)**: Near-monotonic ID columns (e.g., link/jump/containment endpoints) use delta + varint. Other fixed-width integer columns use dense arrays + ZSTD. Rule columns use delta + ZSTD.
- **Booleans (orientations)**: Bit-packed (8 values/byte), up to 8× smaller than byte-per-flag storage.
- **Strings (names, DNA, CIGARs, auxiliary fields)**: Concatenated byte streams with offset arrays, then ZSTD.

**Optional Fields.** (`TAG:TYPE:VALUE`) are encoded by `TYPE`:

- `i` (integer): varint-encoded vector.
- `f` (float): float vector.
- `A` (character): contiguous byte array.
- `Z` (string), `J` (JSON), `H` (hex): concatenated stream with offsets.
- `B` (numeric array): flattened value stream (subtype-dependent, e.g., `c,s,i,f`) + per-row length vector.

Sparse optional fields are handled by sentinel padding: if the $N$-th record lacks a tag, we insert a sentinel value into that column. This preserves $O(1)$ indexing, and the resulting runs of identical sentinels compress effectively under ZSTD.

Table 3 lists the specific columnar structures used for each GFA component.

*3.3.4 Phase 4: ZSTD Compression.* All encoded fields are finally compressed with ZSTD [2]. By this stage, the dominant data (traversals) has already been aggressively reduced by grammar compression and type-specific encoding, so ZSTD operates mostly on compact, low-entropy streams. We use ZSTD at level 9 (configurable via environment variable) to balance throughput and compression ratio.

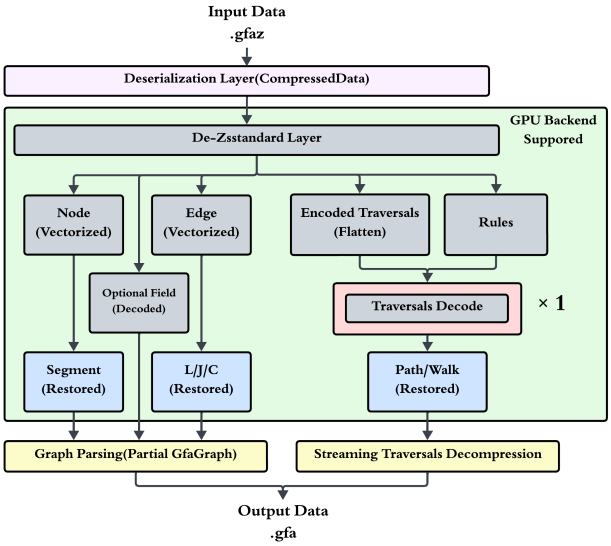



**Figure 3: GFAz decompression workflow. Traversal expansion runs in linear time and is dependency-free across records, enabling batched parallel decode and direct streaming output without rebuilding the full graph in memory.**

## 3.4 Decompression Pipeline

Decompression is partly symmetric to compression. For segments, links, jumps, containments, and optional fields, the decoder simply applies the inverse of the field-specific encodings used in Phase 3 and Phase 4: ZSTD decompression, inverse delta, varint decoding, and orientation bit-unpacking. These field restorations are independent and run in parallel sections.

Traversal decoding is intentionally asymmetric. Rather than rebuilding a full in-memory `GfaGraph`, GFAz expands P/W records with a streaming pipeline. The decoder directly uses `rules_first`, `rules_second`, and `start_id`; no extra rulebook reconstruction step is required. Traversals are partitioned into batches, multiple threads expand and decode one batch at a time, and each batch is written directly as GFA lines to the output file. This design keeps peak memory bounded by the working batch rather than by the total size of all paths and walks.

For each encoded traversal element $e$:

(1) **Check**: If $|e| <$ start_id, it is a literal node ID; we append it directly to the output.
(2) **Expand**: If $|e| \geq$ start_id, it is a rule. We resolve its children via direct array lookup at index ($|e|$ − start_id).
(3) **Recurse**: This lookup mimics a depth-first traversal of the grammar tree. We recursively expand the children until we reach literals. Crucially, if the rule ID $e$ is negative, we expand the children in *reverse order* and flip their signs.

After expansion, inverse delta is applied $D$ times to recover the original node IDs. The recovered oriented IDs are then converted immediately back into the textual P-line or W-line syntax and appended to the output stream. Because traversals are independent, this batched decode-and-write procedure parallelizes naturally across CPU threads. After traversal expansion, the rule table is freed to reclaim memory. This design ensures high throughput. Rule resolution is strictly $O(1)$ (an array read), involving no hash table lookups or complex pointer chasing.

Although the CPU and GPU backends may produce different rule sets and encoded traversal streams, both serialize into the same format (`rulebook + start_id + encoded traversals`), so either backend can correctly decompress files produced by the other.

## 3.5 Incremental Haplotype Extension and Random Access

GFAz supports random access to individual haplotypes and incremental extension by appending new haplotypes. Both operations decode only the lightweight traversal vectors, metadata columns, and stored grammar rules, without reprocessing the full graph.

**Random Access.** The `extract-traversal` operation reconstructs only the requested haplotype. GFAz locates the target path or walk from its metadata columns, slices the corresponding encoded subsequence from the flattened traversal vector, expands the grammar rules, applies inverse delta decoding, and emits only that record.

**Incremental Extension.** The `add-haplotypes` operation appends new paths or walks using the rulebook already stored in the input `.gfaz` file. GFAz validates the new haplotypes, applies the delta transform, encodes only the appended paths or walks, and writes back the updated columns. To preserve decodability, the appended traversal values must remain outside the rule-ID range. Let `start_id` denote the first rule ID stored in the existing file. GFAz verifies that the maximum delta value in the appended traversals is smaller than `start_id`, ensuring that these literals cannot be misinterpreted as grammar rules during later decoding.

## 3.6 GPU Backend

GFAz implements a high-performance GPU backend designed to keep the *traversal compression loop* device-resident. After transferring the traversal stream to GPU memory, all per-round operations—2-mer mining, rule table construction, parallel rule application, and stream compaction—execute on the GPU, avoiding the CPU-side dependency chain that constrains the serial greedy algorithm.

**Parallel 2-mer Mining via Sorting.** The most computationally intensive step is identifying repeated 2-mers. Traditional hash-counting is unsuitable for the GPU due to atomic contention. Instead, we implement a dependency-free, sort-based pipeline given a vector of node IDs $D$ of length $N$:

(1) **GenerateKeys (Canonical 2-mer Keys)**: Since the input size $N$ is known, we allocate a device vector of size $M = N - 1$. A massively parallel kernel launches one thread per index $i \in [0, M)$ and writes a canonical packed 2-mer:

$$K_i = \min(\text{pack}(D_i, D_{i+1}), \text{pack}(-D_{i+1}, -D_i)).$$

This step is embarrassingly parallel and requires no synchronization.
(2) **SortKeys (Group Identicals)**: We sort $K$ on the GPU (radix sort) so that identical 2-mers form contiguous runs.
(3) **MarkDuplicates (Run Membership)**: In parallel, we compute a boolean flag $F_i$ that marks whether $K_i$ belongs to a run,

**Algorithm 2** GPU Parallel 2-mer Mining

**Input:** Device vector $D$ (int32 nodes), integer $N$
**Output:** Device vector *Repeats* (unique int64 2-mers)
$M \leftarrow N - 1$
$Keys \leftarrow$ **ALLOCATE_DEVICE_VECTOR**$(M)$
{1. GenerateKeys: canonical packed 2-mers}
**PARALLEL_FOR** $i \in \{0 \dots M-1\}$
    $K_f \leftarrow \textsc{Pack}(D_i, D_{i+1})$
    $K_r \leftarrow \textsc{Pack}(-D_{i+1}, -D_i)$
    $Keys_i \leftarrow \min(K_f, K_r)$ {canonicalization}
**END_PARALLEL**
{2. SortKeys: group identical 2-mers}
SortKeys($Keys$) {radix sort}
{3. MarkDuplicates: flag run membership}
$Flags \leftarrow$ **ALLOCATE_DEVICE_VECTOR**$(M)$ {uint8}
**PARALLEL_FOR** $i \in \{0 \dots M-1\}$
    $left \leftarrow (i > 0) \wedge (Keys_i = Keys_{i-1})$
    $right \leftarrow (i+1 < M) \wedge (Keys_i = Keys_{i+1})$
    $Flags_i \leftarrow (left \vee right)$
**END_PARALLEL**
{4. CompactUnique: extract repeats and uniquify}
$RawRepeats \leftarrow$ **COMPACT**$(Keys, Flags == 1)$ {copy_if}
$Repeats \leftarrow$ **UNIQUE**$(RawRepeats)$
**return** *Repeats*

**Algorithm 3** GPU Parallel Checkerboard Encoding

**Input:** Device vector $D$ (int32), device hash table $T$
**Output:** New device vector $D'$, per-round `rules_used`
Allocate arrays $F$ (uint8, size $N$) and $V$ (int32, size $N$)
{$F_i$: 0 literal, 1 selected start, 2 consumed}
{1. Candidate marking (no dependencies)}
MarkCandidates$(D, T) \rightarrow (F, V)$
{$F_i \in \{0, 1\}$ after this step}
{2. Checkerboard selection (two kernel-wide phases)}
SelectEvens($F$) {for even $i$ with $F_i$=1: set $F_{i+1}$=2}
ResolveOdds($F$) {for odd $i$ with $F_i$=1: cancel if $F_{i+1}$=1, else set $F_{i+1}$=2}
{3. Emit + compact}
$D' \leftarrow$ ScatterCompact$(D, V, F)$ {emit $V_i$ if $F_i$=1, else $D_i$; skip $F_i$=2}
MarkUsedRules$(F, V) \rightarrow$ `rules_used`

using only neighbor comparisons:

$$F_i = \begin{cases} (K_0 = K_1), & i = 0, \\ (K_i = K_{i-1}) \ \vee \ (K_i = K_{i+1}), & 0 < i < M-1, \\ (K_{M-1} = K_{M-2}), & i = M-1. \end{cases}$$

(4) **CompactUnique (Extract Repeats)**: We apply stream compaction (`copy_if`) to extract all keys with $F_i = 1$, then apply `unique` to produce the final set of repeated 2-mers (frequency $\geq 2$) used as candidate rules for this round.

**Parallel Checkerboard Encoding.** In the CPU backend, rule substitution is a serial, greedy scan: a replacement at index $i$ consumes $(i, i+1)$ and therefore blocks a replacement starting at $i+1$. This induces a strict left-to-right dependency chain. On the GPU, we remove this dependency with a two-phase "checkerboard" schedule that selects a non-overlapping set of rule starts using only local neighbor state and two global phases (even pass, then odd pass):

(1) **MarkCandidates (Lookup + Orientation)**: For each $i \in [0, N-2]$, one thread forms the canonical key $K_i$ and performs a hash-table lookup in $T$. If found, let $r_i = T[K_i]$ be the rule ID assigned to this 2-mer key; we mark $i$ as a candidate start and store the signed replacement value: $+r_i$ if $\text{pack}(D_i, D_{i+1}) = K_i$, else $-r_i$.
(2) **SelectEvens**: All even candidates $i \in \{0, 2, 4, \dots\}$ are selected simultaneously. Because intervals $[2k, 2k+1]$ do not overlap, we can safely mark each right neighbor $i$+1 as *consumed*.
(3) **ResolveOdds**: Odd candidate starts $i \in \{1, 3, 5, \dots\}$ are selected only if their right neighbor $i$+1 is *not* an even start. If $i$+1 is an even start, the odd candidate is canceled; otherwise it is selected and consumes $i$+1.
(4) **ScatterCompact**: After selection, $F_i$ encodes three states: literal (0), selected start (1), or consumed (2). We keep indices not marked consumed, emitting the signed rule ID at selected starts and the original literal otherwise. A `rules_used` bitmap is built in the same pass.

**Compact, Sort, and Remap.** Following encoding, we perform the same optimization steps as the CPU backend but strictly using GPU primitives. We use a parallel prefix sum (scan) to calculate the destination indices for active rules (compaction). The rules are then sorted by their 2-mer keys using GPU Radix Sort to ensure optimal delta encoding. Finally, a parallel kernel remaps all rule occurrences in the traversal vector to their new sorted IDs. The final output matches the CPU pipeline exactly: a single flattened `int32` traversal vector, two parallel rule definition vectors, and `start_id`, allowing for identical downstream processing.

**Field Encoding.** Once traversals and rule tables are produced as contiguous device vectors, we compress them directly on the GPU using `nvComp` ZSTD. For boolean columns (e.g., orientations), we first bit-pack flags (1 bit/value) and then apply ZSTD; for integer and byte streams we apply ZSTD directly, leveraging the structure created by sorting, delta encoding, and compaction.

**GPU Decompression (Traversals).** A direct GPU port of CPU-style recursive expansion (depth-first rule chasing) performs poorly: varying grammar depth introduces divergence, and repeated rule lookups lead to irregular memory access. We therefore use a *rule pre-expansion* strategy that converts recursive decoding into a small number of dense, regular memory passes:

(1) **PreExpandRules (Once per Rulebook)**: We pre-expand every rule into a contiguous buffer. Concretely, we compute each rule's final expansion length (number of literals after full expansion), exclusive-scan these lengths to obtain per-rule offsets, then materialize a dense array that stores each rule's fully expanded literal sequence. Negative rule references are handled by reversing child order and flipping signs during expansion.
(2) **ComputeOffsets (Per Traversal)**: For each element of the encoded traversal, we compute its output length (1 for raw node IDs; `rule_sizes`$[|r| - \text{min\_rule\_id}]$ for rules) and perform an exclusive scan to obtain the exact output write position for every element.

**Table 4: Evaluation datasets.**

| Name | Type | Size |
|---|---|---|
| `chr1.pan.smooth.gfa` | Human chr (smooth) | 7.1 GB |
| `chr6.pan.smooth.gfa` | Human chr (smooth) | 3.8 GB |
| `EcoliGraph_MGC.gfa` | Microbial (*E. coli*) | 0.113 GB |
| `hprc-v1.1-mc-chm13.gfa` | Human WGS (HPRC) | 48 GB |
| `hprc-v2.0-mc-chm13.gfa` | Human WGS (HPRC) | 358 GB |
| `hprc-v2.1-mc-chm13.gfa` | Human WGS (HPRC) | 369 GB |

(3) **ExpandTraversalSinglePass (Scatter/Copy)**: A single kernel writes the decoded traversal into the output buffer: literals are copied directly; rules are expanded by bulk-copying their pre-expanded segments from the dense rule buffer. If a rule ID is negative, we copy the segment sequence in reverse order and negate each element.

Compared to an alternative reverse-$n$-round decoder that undoes grammar layers one level at a time (which requires repeated full-vector *scan* → *expand* → *re-scan* passes and global synchronization), this pre-expansion strategy removes the bottleneck of multiple kernel launches. Variable-depth recursion is paid once per rulebook, and traversal decoding becomes a small constant number of regular passes. This increases throughput from about 30 GB/s (using our deprecated iterative expander) to over 300 GB/s. In practice, the decode phase is bandwidth-dominated, followed by GPU-resident inverse delta (prefix sum) to recover original node IDs. Other fields are decoded and copied back to host memory.

## 4 Evaluation

### 4.1 Experimental Setup

All experiments run on a workstation with an AMD Ryzen Threadripper PRO 9955WX (16 cores), an NVIDIA RTX Pro 6000 GPU, and 512 GB of DDR5 6400 MHz memory. For CPU compressors that support multithreading, we enable all 16 hardware threads. Unless otherwise noted, `zstd` uses compression level 9. To measure algorithmic throughput rather than storage performance, we report *warm-cache* timings averaged over 5 runs: before each run, we prime the OS page cache by streaming the input file once (e.g., `cat input.gfa > /dev/null`) to minimize disk I/O effects.

### 4.2 Dataset

We evaluate our compressor on the datasets summarized in Table 4. For readability, we refer to the chromosome-scale smoothed graphs using short names (`chr1.pan.smooth.gfa` and `chr6.pan.smooth.gfa`) [10]. These datasets are representative for three reasons. The chromosome scale PGGB graphs (chr1/chr6) provide realistic human pangenome structure at a manageable scale, making it easier to stress specific components (e.g., traversal redundancy) without the full whole-genome footprint. Second, the HPRC minigraph–cactus graphs span multiple rapidly evolving public releases (v1.1, v2.0, v2.1) [10], allowing us to evaluate robustness as graph construction pipelines and graph complexity change over time. Finally, `EcoliGraph_MGC.gfa` [15] tests generality beyond human genomes on a microbial pangenome with dense path sets and different repeat/variation characteristics.

**Table 5: Compression ratio, compression (Co.) and decompression (De.) throughput (MB/s) across datasets. Blue indicates the best performance, green denotes the second best.**

| Dataset | metrics | Gzip | Zstd | sqz | sqz+bgzip | sqz+Zstd | GBZ | GFAz (CPU) | GFAz (GPU) |
|---|---|---|---|---|---|---|---|---|---|
| chr1. | Ratio | 5.59 | 7.54 | 3.09 | 18.0 | 16.7 | 9.52 | 35.4 | 31.7 |
| | Co. | 46.2 | 2178 | 3.95 | 3.97 | 4.00 | 12.1 | 385 | 2754 |
| | De. | 359 | 1618 | 21.6 | 21.4 | 21.2 | 284 | 1658 | 8124 |
| chr6. | Ratio | 5.04 | 6.99 | 5.51 | 20.8 | 19.2 | 19.2 | 35.4 | 28.18 |
| | Co. | 41.0 | 1712 | 3.56 | 3.56 | 3.59 | 10.7 | 348 | 3791 |
| | De. | 348 | 1515 | 20.1 | 20.4 | 20.2 | 281 | 1758 | 7230 |
| E.coli | Ratio | 4.69 | 5.67 | 1.26 | 7.46 | 6.79 | 5.58 | 18.3 | 16.7 |
| | Co. | 33.3 | 1356 | 4.57 | 4.53 | 4.62 | 20.2 | 190 | 678 |
| | De. | 310 | 1258 | 34.0 | 32.2 | 34.5 | 197 | 491 | 1430 |
| HPRCv1.1 | Ratio | 4.02 | 5.32 | - | - | - | 14.0 | 22.4 | 20.4 |
| | Co. | 36.4 | 1657 | - | - | - | 84.5 | 231 | 4843 |
| | De. | 319 | 1234 | - | - | - | 650 | 1058 | 9435 |
| HPRCv2.0 | Ratio | 4.19 | 6.49 | - | - | - | 66.8 | 83.9 | 76.4 |
| | Co. | 49.1 | 1514 | - | - | - | 130 | 367 | - |
| | De. | 342 | 1240 | - | - | - | 648 | 1652 | - |
| HPRCv2.1 | Ratio | 4.19 | 6.43 | - | - | - | 64.2 | 82.8 | 74.2 |
| | Co. | 48.9 | 1540 | - | - | - | 136 | 348 | - |
| | De. | 343 | 1241 | - | - | - | 652 | 1559 | - |

### 4.3 Compression-Decompression Performance

We include gzip and zstd as general-purpose baselines, GBZ as a graph-native compressed format, and SQZ as a pangenome-oriented compressor; for SQZ we also report `sqz+bgzip` and `sqz+zstd` to probe its best ratio under alternative back-end compressors. SQZ runs out of memory on HPRC v1.1 and larger graphs, so those entries are marked NA. SQZ and GBZ also drop parts of the original GFA representation, making the GFAz comparisons conservative. For the 300+ GB HPRC v2.0/v2.1 graphs, the current GPU implementation runs out of device memory; we therefore report GPU ratios via a CPU simulation of the GPU rule-mining/encoding logic and omit GPU throughput for those two datasets. Table 5 reveals three consistent patterns.

**Insight 1 (ratio gains increase with graph complexity).** GFAz gives the best ratio on all datasets with available results, for both backends. On chr1, GFAz (CPU) reaches 35.4× versus zstd 7.54× and GBZ 9.52×; on chr6, 35.4× versus 6.99× and 19.2×. On larger HPRC graphs, the gap widens: on v2.0, GFAz (CPU) reaches 83.9× versus zstd 6.49× and GBZ 66.8×; on v2.1, 82.8× versus 6.43× and 64.2×. The GPU ratio is consistently but modestly below CPU (e.g., 31.7× vs 35.4× on chr1, 20.4× vs 22.4× on v1.1), consistent with

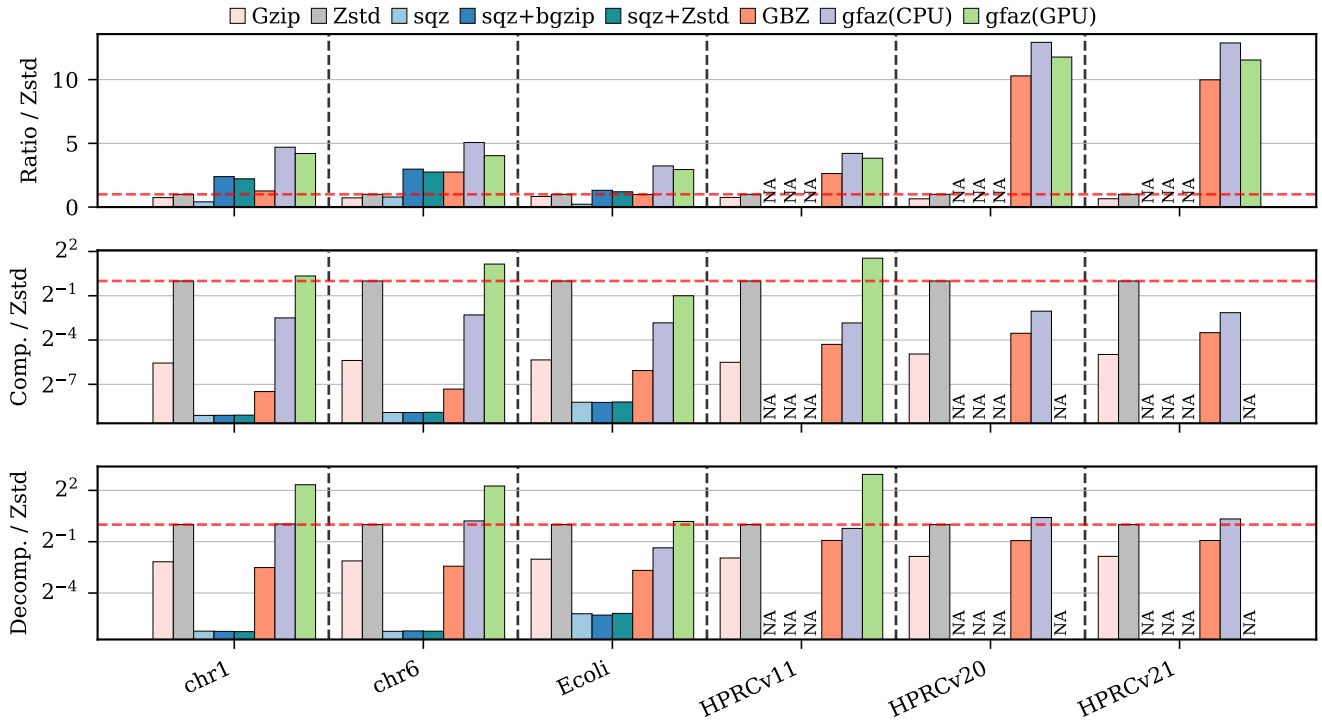



**Figure 4: Normalized compression ratio, compression speed, and decompression speed relative to Zstd across all datasets. Results are normalized such that Zstd = 1.0 (dashed horizontal line). Speed metrics are plotted on a log scale (base 2).**

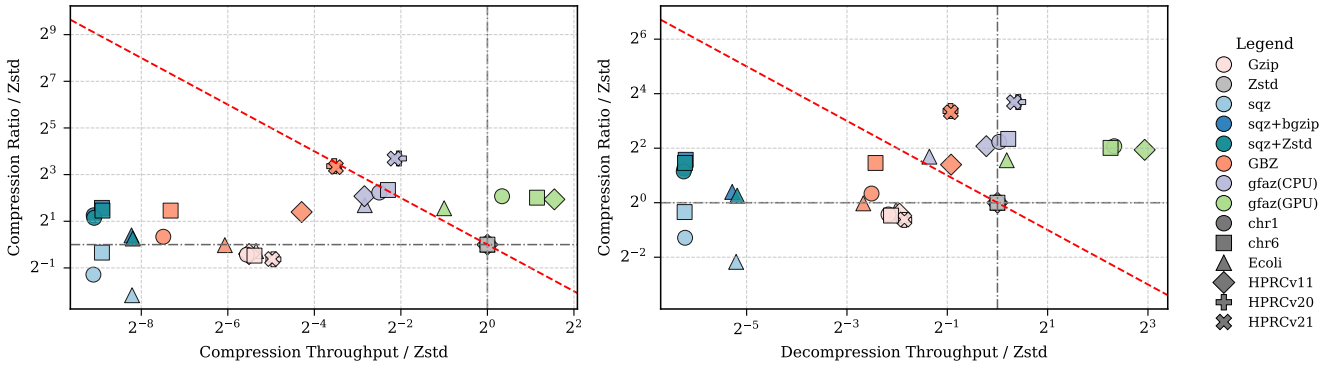



**Figure 5: Pareto trade-off between compression ratio (y-axis) and speed (x-axis), normalized to Zstd (baseline at 1.0). The dashed diagonal line ($y = 1/x$) represents an iso-efficiency curve where a 2× increase in ratio compensates for a 2× drop in speed. Points above this line and to the right represent better trade-offs than the baseline.**

checkerboard encoding (Section 3.6) replacing fewer overlapping opportunities than serial greedy encoding in each round.

**Insight 2 (Throughput remains strong across CPU/GPU).** When inputs fit GPU memory, GFAz (GPU) is the fastest high-ratio operating point: on HPRC v1.1, it reaches 4843/9435 MB/s compression/decompression, versus GFAz (CPU) at 231/1058 MB/s and zstd at 1657/1234 MB/s. On chr1 and chr6, GFAz (GPU) also leads throughput, showing that the GPU backend is most effective when traversal data can remain device-resident. For v2.0/v2.1, GPU throughput is unavailable because the traversal stream exceeds device memory, but GFAz (CPU) still gives the fastest decompression (1652 and 1559 MB/s) while retaining much higher ratios than zstd. CPU compression is slower than zstd on these largest graphs, but this trade-off is favorable for storage and repeated-read workloads. On the small *E. coli* input, zstd compresses fastest, while GFAz (GPU) still improves both ratio and decompression speed.

**Table 6: Ablation of `Compact` and `Sort` on rulebook effectiveness. Each cell shows $H$ / $L$, where $H = r \gg 32$ and $L = r \,\&\, \texttt{0xffffffff}$; each entry is the ZSTD-compressed size followed by the compression ratio in parentheses.**

| Dataset | compact+sort | compact | sort | none |
|---|---|---|---|---|
| EcoliGraph_MGC | 53.9 KB (**9.6**×) / 215 KB (**2.4**×) | 348 KB (1.5×) / 358 KB (1.4×) | 70.7 KB (9.6×) / 291 KB (2.3×) | 463 KB (1.5×) / 477 KB (1.4×) |
| chr1.pan.smooth | 4.3 MB (**11.2**×) / 30.4 MB (**1.6**×) | 40.6 MB (1.2×) / 40.6 MB (1.2×) | 5.2 MB (11.1×) / 38.0 MB (1.5×) | 50.0 MB (1.2×) / 50.0 MB (1.2×) |
| chr6.pan.smooth | 2.0 MB (**12.1**×) / 14.0 MB (**1.7**×) | 19.3 MB (1.2×) / 19.3 MB (1.2×) | 2.4 MB (12.0×) / 17.8 MB (1.6×) | 24.1 MB (1.2×) / 24.1 MB (1.2×) |
| hprc-v1.1 | 61.2 MB (**10.2**×) / 330 MB (**1.9**×) | 565 MB (1.1×) / 561 MB (1.1×) | 73.9 MB (10.4×) / 418 MB (1.8×) | 697 MB (1.1×) / 692 MB (1.1×) |
| hprc-v2.0 | 124 MB (**13.1**×) / 1254 MB (**1.3**×) | 1512 MB (1.1×) / 1513 MB (1.1×) | 153 MB (13.0×) / 1558 MB (1.3×) | 1857 MB (1.1×) / 1859 MB (1.1×) |
| hprc-v2.1 | 130 MB (**13.1**×) / 1315 MB (**1.3**×) | 1588 MB (1.1×) / 1589 MB (1.1×) | 161 MB (12.9×) / 1632 MB (1.3×) | 1949 MB (1.1×) / 1951 MB (1.1×) |

**Table 7: Random-access evaluation using `extract-traversal`. We measure the time and peak memory required to reconstruct only 1, 4, or 16 selected paths/walks from an existing `.gfaz` file.**

| Dataset | Extracted | Elapsed (s) | Peak Mem. (GB) | GFA Size (GB) |
|---|---|---|---|---|
| chr1 | 1 | 0.102 | 0.38 | 7.1 |
| | 4 | 0.103 | 0.37 | 7.1 |
| | 16 | 0.106 | 0.41 | 7.1 |
| hprc-v1.1 | 1 | 1.006 | 4.09 | 48 |
| | 4 | 1.012 | 4.10 | 48 |
| | 16 | 1.029 | 4.13 | 48 |
| hprc-v2.0 | 1 | 2.285 | 9.26 | 358 |
| | 4 | 2.302 | 9.28 | 358 |
| | 16 | 2.312 | 9.30 | 358 |
| hprc-v2.1 | 1 | 2.278 | 9.65 | 369 |
| | 4 | 2.303 | 9.67 | 369 |
| | 16 | 2.380 | 9.74 | 369 |

**Table 8: Incremental haplotype extension using `add-haplotypes`. Each run appends 50 new paths/walks using the rulebook already stored in the input `.gfaz` file.**

| Dataset | Added Haplotypes | Elapsed (s) | Peak Memory (GB) | GFA Size (GB) |
|---|---|---|---|---|
| chr1 | 50 | 0.95 | 0.97 | 7.1 |
| chr6 | 50 | 0.54 | 0.51 | 3.8 |
| hprc-v1.1 | 50 | 3.25 | 3.25 | 48 |
| hprc-v2.0 | 50 | 9.80 | 16.85 | 358 |
| hprc-v2.1 | 50 | 10.20 | 17.32 | 369 |

**Insight 3 (alternative methods occupy weaker ratio–speed fronts).** SQZ variants can improve ratio over gzip/zstd on chromosome graphs, but their throughput (about 3–5 MB/s compression and about 20–35 MB/s decompression) is two orders of magnitude lower than GFAz. GBZ is the strongest non-GFAz ratio baseline on large HPRC graphs, but remains below GFAz in ratio (66.8×/64.2× vs 83.9×/82.8× on v2.0/v2.1) and decompression speed (648/652 vs 1652/1559 MB/s on CPU), while preserving fewer GFA semantics.

**Normalized and Pareto views.** Figures 4 and 5 summarize the same results normalized to Zstd. GFAz consistently improves decompression and ratio efficiency. For compression, only the small input can fall below the line but still keeps a substantial multi-fold ratio advantage over Zstd.

## 4.4 Random Access and Incremental Extension

We next evaluate the two traversal-centric operations enabled by the `.gfaz` layout: extracting a small number of haplotypes without full decompression, and appending new haplotypes without recompressing the entire file.

**Random access.** Table 7 reports extraction of 1, 4, or 16 paths/walks from existing `.gfaz` files. Runtime stays nearly flat as more records are requested. On HPRC v2.1 (369 GB), extracting 1, 4, and 16 haplotypes takes 2.28 s, 2.30 s, and 2.38 s, respectively. Peak memory also changes only slightly, from 9.65 GB to 9.74 GB.

**Incremental extension.** Table 8 reports the cost of appending 50 new haplotypes. This takes 0.54–0.95 s on chromosome-scale graphs, 3.25 s on HPRC v1.1, and about 10 s on the 358–369 GB graphs. On HPRC v2.1, appending 50 haplotypes takes 10.2 s, compared with roughly 1100 s for full recompression.

## 4.5 Ablation: Compact and Sort

We evaluate four workflow variants: `compact+sort` (full design), `compact` only, `sort` only, and `none`. For each packed 64-bit rule ID $r$, we split it into two 32-bit integer streams: $H = r \gg 32$ (high 32 bits) and $L = r \,\&\, \texttt{0xffffffff}$ (low 32 bits) during compression, then entropy-code each stream with ZSTD. Results directly validate the design rationale in Section 3.3. Sorting is the dominant factor for $H$: on chr1, $H$ improves from 1.2× (`none`) to 11.1× (`sort`), and to 11.2× with `compact+sort`; similarly on hprc-v2.1, 1.1× (`none`) to 12.9× (`sort`) and 13.1× (`compact+sort`). Compact then provides additional gains by removing unused rules and shrinking emitted streams: for chr1 $L$ improves from 1.5× (`sort`) to 1.6× (`compact+sort`), and for hprc-v2.1 from 1.3× to 1.3× with a concrete size drop (1632 MB to 1315 MB). Compared with `compact` only, the full `compact+sort` pipeline is consistently much better on $H$ (e.g., chr6: 12.1× vs 1.2×), showing that compacting without ordering does not create enough entropy structure for the rulebook. This also clarifies a weakness of SQZ: although SQZ is grammar-based, highly variable rule symbols without strong ordering reduce rule-stream regularity, making entropy coding less effective than the sorted rulebook layout used by GFAz.

## 4.6 Performance Breakdown

Tables 10 and 9 break down stage-level profiling for the GPU and CPU backends respectively. Each table reports a `Grammar subtotal` (traversal-related stages) and `Others` (all remaining fields—segment sequences, link/jump/containment columns, optional fields, and their associated ZSTD encoding).

On both backends, decompression is dominated by `Others`: 87.9% on CPU and 86.6% on GPU, leaving only 12.1% (CPU) and 13.4% (GPU) in grammar stages. Within grammar decode, traversal expansion itself is fast (371.93 ms at 6.06 GB/s on CPU; 24.73 ms at 91.2 GB/s on GPU), consistent with one-pass expansion and $O(1)$ rule lookup (`rule_id - min_rule_id`). This supports the design

**Table 9: CPU stage-level profiling for compression and decompression. Share is computed as `time / stage total`.**

| Stage | Time (ms) | Tput (GB/s) | Share |
|---|---|---|---|
| *Compression (total: 19888.78 ms)* | | | |
| Grammar subtotal | 11004.68 | 0.20 | 55.3% |
| generate_rules | 6000.89 | 0.38 | 30.2% |
| encode_traversals | 4785.13 | 0.47 | 24.1% |
| compact_sort_remap | 196.83 | 11.5 | 1.0% |
| split+delta_encode | 27.93 | 3.30 | 0.1% |
| zstd compress | 467.58 | 0.47 | 2.4% |
| Others | 8884.10 | – | 44.7% |
| *Decompression (total: 4221.47 ms)* | | | |
| Grammar subtotal | 511.52 | 4.32 | 12.1% |
| zstd decompress | 102.39 | 1.60 | 2.4% |
| decode rules (prefix sum) | 2.85 | 32.4 | 0.1% |
| expand traversals | 371.93 | 6.06 | 8.8% |
| decode traversals (pfx sum) | 34.35 | 65.6 | 0.8% |
| Others | 3709.95 | – | 87.9% |

**Table 10: GPU stage-level profiling for compression and decompression. Share is computed as `time / stage total`.**

| Stage | Time (ms) | Tput (GB/s) | Share |
|---|---|---|---|
| *Compression (total: 2322.85 ms)* | | | |
| Grammar subtotal | 571.85 | 3.96 | 24.6% |
| find_max_abs | 5.96 | 378.0 | 0.3% |
| delta_encode | 5.64 | 400.0 | 0.2% |
| find_repeated_2mers | 195.59 | 11.5 | 8.4% |
| create_rule_table | 18.36 | 122.8 | 0.8% |
| apply_2mer_rules | 105.24 | 21.4 | 4.5% |
| compact_rules_remap | 25.68 | 87.8 | 1.1% |
| sort_rules_remap | 16.70 | 135.0 | 0.7% |
| split+delta_encode | 0.90 | 102.2 | 0.0% |
| Zstd compress (nvComp) | 197.78 | 11.4 | 8.5% |
| Others | 1751.00 | – | 75.4% |
| *Decompression (total: 876.00 ms)* | | | |
| Grammar subtotal | 117.00 | 19.3 | 13.4% |
| Zstd decompress (nvComp) | 85.85 | 26.3 | 9.8% |
| decode rules (prefix sum) | 0.44 | 367.2 | 0.1% |
| expand traversals | 24.73 | 91.2 | 2.8% |
| decode traversals (pfx sum) | 5.98 | 377.1 | 0.7% |
| Others | 759.00 | – | 86.6% |

claim that traversal reconstruction is not the end-to-end bottleneck once the rulebook is available.

The two backends differ mainly in compression cost distribution. On CPU, grammar stages consume 55.3% of compression time, with `generate_rules` (30.2%) and `encode_traversals` (24.1%) as dominant kernels. On GPU, grammar drops to 24.6% of total compression time (`find_repeated_2mers` 8.4%, `apply_2mer_rules` 4.5%), while `Others` rises to 75.4%, consistent with nvComp/ZSTD orchestration overhead in the current pipeline. This profile explains why the GPU grammar algorithm is already fast but end-to-end speedups are now limited by entropy and non-traversal stages.

**Table 11: CPU peak resident memory using 16 threads for compression, a legacy decompression baseline that fully materializes the in-memory `GfaGraph`, and the implemented streaming decompression path.**

| Dataset | Comp. Peak (GB) | Full-Graph Decomp. (GB) | Streaming Decomp. (GB) | GFA Size (GB) |
|---|---|---|---|---|
| chr1 | 7.4 | 6.3 | 0.98 | 7.1 |
| chr6 | 3.2 | 3.0 | 0.66 | 3.8 |
| Ecoli | 0.24 | 0.18 | 0.18 | 0.113 |
| hprc-v1.1 | 45 | 37 | 3.4 | 48 |
| hprc-v2.0 | 343 | 289 | 19.3 | 358 |
| hprc-v2.1 | 356 | 295 | 20.7 | 369 |

## 4.7 CPU Memory Profiling

Table 11 compares three CPU memory profiles measured with 16 threads: compression, a legacy decompression baseline that fully materializes the entire `GfaGraph`, and the implemented streaming decompression path that expands one path/walk and writes it out immediately. The fully materialized baseline reaches 37 GB, 289 GB, and 295 GB on HPRC v1.1, v2.0, and v2.1, respectively. In contrast, the implemented streaming path reduces peak decompression memory to 3.4 GB, 19.3 GB, and 20.7 GB, a reduction of about 14–15×.

High compression peak memory is less critical in practice for three reasons. First, it occurs primarily in the first rule-encoding round and drops quickly afterward as the traversal stream shrinks. Second, compression is usually a one-time preprocessing step, whereas decompression and downstream analyses are run repeatedly. Third, downstream graph tools such as VG [5] and ODGI [7] already require memory footprints larger than the input GFA size.

## 5 Conclusion and Future Work

We presented GFAz, a high-performance GFA compressor that combines orientation-aware 2-mer grammar compression with a columnar, type-specific encoding of all GFA record types. By exploiting the canonical symmetry of forward and reverse node orientations, GFAz collapses traversal redundancy in linear time across iterative grammar rounds. Fully parallel CPU (OpenMP) and GPU (CUDA/nvComp) backends deliver state-of-the-art compression ratios (20–80×) and throughput on datasets ranging from microbial pangenomes to whole-genome human graphs.

Our current GPU backend requires the full traversal stream to fit in device memory, which prevents processing the largest graphs (300+ GB) on current hardware. In future work, we plan to introduce memory-aware scheduling for both CPU and GPU pipelines by partitioning traversals into streaming tiles that can be compressed and decompressed independently within a fixed memory budget to eliminate out-of-memory failures regardless of input size.

## Acknowledgments

We would like to acknowledge the Human Pangenome Reference Consortium (BioProject ID: PRJNA730823) and its funder, the National Human Genome Research Institute (NHGRI). This work was partially supported by the National Institutes of Health (NIH) under Award Number R01GM157443.

## References


[1] James K Bonfield. 2022. CRAM 3.1: advances in the CRAM file format. *Bioinformatics* 38, 6 (2022), 1497–1503.

[2] Yann Collet. 2021. RFC 8878: Zstandard Compression and the'application/zstd'Media Type.

[3] Jordan M Eizenga, Adam M Novak, Jonas A Sibbesen, Simon Heumos, Ali Ghaffaari, Glenn Hickey, Xian Chang, Josiah D Seaman, Robin Rounthwaite, Jana Ebler, et al. 2020. Pangenome graphs. *Annual review of genomics and human genetics* 21, 1 (2020), 139–162.

[4] Erik Garrison, Andrea Guarracino, Simon Heumos, Flavia Villani, Zhigui Bao, Lorenzo Tattini, Jörg Hagmann, Sebastian Vorbrugg, Santiago Marco-Sola, Christian Kubica, et al. 2024. Building pangenome graphs. *Nature Methods* 21, 11 (2024), 2008–2012.

[5] Erik Garrison, Jouni Sirén, Adam M Novak, Glenn Hickey, Jordan M Eizenga, Eric T Dawson, William Jones, Shilpa Garg, Charles Markello, Michael F Lin, et al. 2018. Variation graph toolkit improves read mapping by representing genetic variation in the reference. *Nature biotechnology* 36, 9 (2018), 875–879.

[6] GFA Specification Working Group. 2023. GFA1: Graphical Fragment Assembly Format Specification. https://gfa-spec.github.io/GFA-spec/GFA1.html. Accessed: 2026-02-08.

[7] Andrea Guarracino, Simon Heumos, Sven Nahnsen, Pjotr Prins, and Erik Garrison. 2022. ODGI: understanding pangenome graphs. *Bioinformatics* 38, 13 (2022), 3319–3326.

[8] Peter Heringer and Daniel Doerr. 2025. Human readable compression of GFA paths using grammar-based code. *bioRxiv* (2025), 2025–05.

[9] Glenn Hickey, Jean Monlong, Jana Ebler, Adam M Novak, Jordan M Eizenga, Yan Gao, Tobias Marschall, Heng Li, and Benedict Paten. 2024. Pangenome graph construction from genome alignments with Minigraph-Cactus. *Nature biotechnology* 42, 4 (2024), 663–673.

[10] Human Pangenome Reference Consortium. 2025. Human Pangenome Project Data Portal. https://42basepairs.com/browse/s3/human-pangenomics/pangenomes. Public HPRC pangenome graph datasets hosted via the 42basepairs platform; accessed February 8, 2026.

[11] Human Pangenome Reference Consortium. 2025. Human Pangenome Reference Consortium: Release Timeline. https://humanpangenome.org/release-timeline/. Accessed: February 8, 2026.

[12] Heng Li, Bob Handsaker, Alec Wysoker, Tim Fennell, Jue Ruan, Nils Homer, Gabor Marth, Goncalo Abecasis, Richard Durbin, and 1000 Genome Project Data Processing Subgroup. 2009. The sequence alignment/map format and SAMtools. *bioinformatics* 25, 16 (2009), 2078–2079.

[13] Wen-Wei Liao, Mobin Asri, Jana Ebler, Daniel Doerr, Marina Haukness, Glenn Hickey, Shuangjia Lu, Julian K Lucas, Jean Monlong, Haley J Abel, et al. 2023. A draft human pangenome reference. *Nature* 617, 7960 (2023), 312–324.

[14] Mao-Jan Lin, Sheila Iyer, Nae-Chyun Chen, and Ben Langmead. 2024. Measuring, visualizing, and diagnosing reference bias with biastools. *Genome Biology* 25, 1 (2024), 101.

[15] Nina Marthe and Francois Sabot. 2025. Data used for the analysis described in GrAnnoT paper. doi:10.23708/DO1RTF

[16] NVIDIA Corporation. 2026. NVIDIA/nvcomp: Repository for nvCOMP docs and examples. https://github.com/NVIDIA/nvcomp. GitHub repository, accessed February 9, 2026.

[17] Simona Secomandi, Guido Roberto Gallo, Riccardo Rossi, Carlos Rodríguez Fernandes, Erich D Jarvis, Andrea Bonisoli-Alquati, Luca Gianfranceschi, and Giulio Formenti. 2025. Pangenome graphs and their applications in biodiversity genomics. *Nature Genetics* 57, 1 (2025), 13–26.

[18] Jouni Sirén, Erik Garrison, Adam M Novak, Benedict Paten, and Richard Durbin. 2020. Haplotype-aware graph indexes. *Bioinformatics* 36, 2 (2020), 400–407.

[19] Jouni Sirén, Jean Monlong, Xian Chang, Adam M Novak, Jordan M Eizenga, Charles Markello, Jonas A Sibbesen, Glenn Hickey, Pi-Chuan Chang, Andrew Carroll, et al. 2021. Pangenomics enables genotyping of known structural variants in 5202 diverse genomes. *Science* 374, 6574 (2021), abg8871.

[20] Jouni Sirén and Benedict Paten. 2022. GBZ file format for pangenome graphs. *Bioinformatics* 38, 22 (2022), 5012–5018.

[21] Ting Wang, Lucinda Antonacci-Fulton, Kerstin Howe, Heather A Lawson, Julian K Lucas, Adam M Phillippy, Alice B Popejoy, Mobin Asri, Caryn Carson, Mark JP Chaisson, et al. 2022. The Human Pangenome Project: a global resource to map genomic diversity. *Nature* 604, 7906 (2022), 437–446.

[22] Ryan R. Wick, Mark B. Schultz, Justin Zobel, and Kathryn E. Holt. 2015. Bandage: Interactive visualization of de novo genome assemblies. *Bioinformatics* 31, 20 (2015), 3350–3352. doi:10.1093/bioinformatics/btv383

[23] Taolue Yang, Youyuan Liu, Bo Jiang, and Sian Jin. 2024. GPUFASTQLZ: An Ultra Fast Compression Methodology for Fastq Sequence Data on GPUs. In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 317–325.

[24] Vilém Zouhar, Clara Meister, Juan Gastaldi, Li Du, Tim Vieira, Mrinmaya Sachan, and Ryan Cotterell. 2023. A formal perspective on byte-pair encoding. In *Findings of the Association for Computational Linguistics: ACL 2023*. 598–614.